GEBZE TECHNICAL UNIVERSITY

Computer Engineering Department

CSE 396 Module Documentation

Çılgın Kurye

Module M2: Network and Synchronization

Company: Board&Byte **Group:** Group-8 **Date:** 7 November 2026

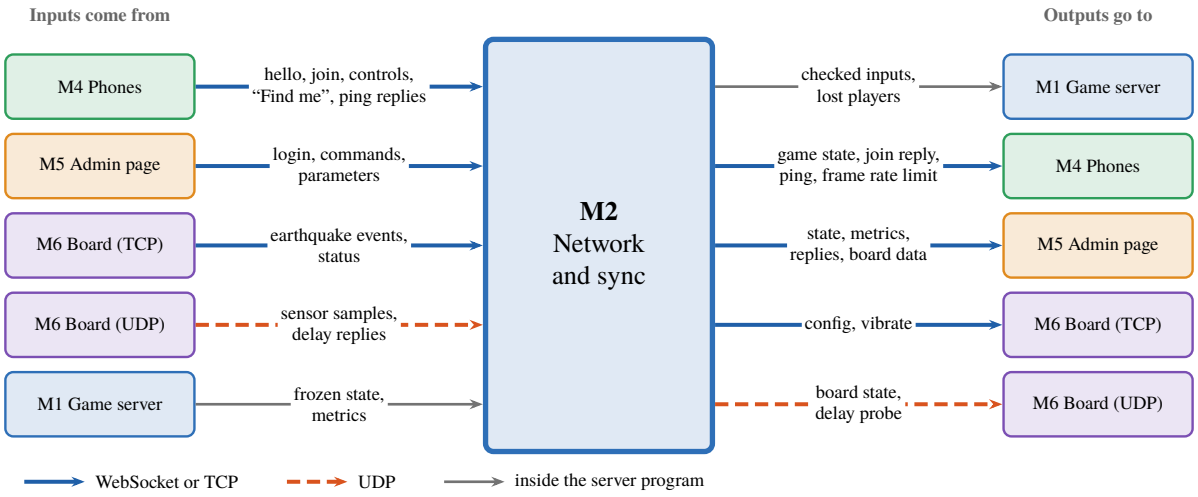
Name	Student No.	Name	Student No.
Tuğberk Akbulut	230104004070	Ayşe Poyraz	230104004085
Hasan Hüseyin Özdemir	230104004048	Umutcan Ocak	220104004903

1. Purpose

M2 connects the phones, the admin page and the LED board to the game server. It does not make game decisions. It takes messages from outside, checks them and gives them to M1. After each tick it takes the state from M1 and sends it to everybody. M2 also makes sure that nothing from outside can change the game state directly. For the admin, it measures the network and it can make the network worse on purpose for tests.

Runs on	Technology	Requirements
Main computer, in the same server program as M1	TypeScript on Node.js, WebSocket for phones and admin, TCP and UDP for the LED board, MessagePack binary messages, QR code for joining	R1 R9 R10 R11

M2 is the only module with links to the outside. The figure shows every neighbour and the kind of link that each message uses.



2. Requirements covered

M2 takes part in four requirements. The last column says what will work when M2 is finished.

ID	Requirement	What M2 does for it	When M2 is done
R1	Multiplayer game	Up to 8 phones join with a QR code or a six digit game code, only with the browser	Players join in seconds without installing anything
R9	Live admin panel	Admin channel for commands, parameters, state, metrics and board data	The admin page controls the game from any computer on the game Wi-Fi
R10	Network and measurement	Ping and jitter for each phone, real loss on the board link. Added delay, jitter and loss for each player and for the board	The admin can make the network worse and see the effect live
R11	Engineering requirements	Inputs are checked and queued. Every phone gets the same frozen state of each tick	No phone can change the game by itself, and all players see the same game

3. Inputs and outputs

The table lists every message group with its link. All messages are MessagePack, a compact binary form of JSON.

Direction	Data	From / to	Content
In	Phone messages	M4, WebSocket	Hello with player key, name and game code, join with team, controls, “Find me”, ping replies
In	Admin messages	M5, WebSocket	Commands, parameter changes, requests for parameters and the game log
In	Board messages	M6, TCP and UDP	Earthquake events and status on TCP. Sensor samples and delay replies on UDP
In	State and metrics	M1	Frozen game state and board state every tick, metrics twice per second
Out	To M1	M1	Checked controls, joins, “Find me”, admin commands, board events, lost and returning players, network metrics
Out	To phones	M4, WebSocket	Game state 30 times per second, join replies, ping, frame rate limit
Out	To admin	M5, WebSocket	State 10 times per second, metrics, replies, board data
Out	To board	M6, TCP and UDP	Config and vibrate commands on TCP. Board state and delay probes on UDP

4. Main components

M2 is made of eight parts. Each one takes some data and gives a result to the next part.

Component	Takes	Gives
Web server and join	Page requests from phones and the admin	The phone and admin pages, the 3D models, the city plan, the QR code and the game code
Sessions and access	New links, player keys	One session for each phone and one admin. A player key keeps the same motorcycle after a reload
Codec and checker	Raw messages	Checked messages. A bad message gets an error reply. Too many or too large messages are dropped
Input queue	Checked inputs	The inputs of one tick for M1
State broadcaster	Frozen state from M1	The same encoded bytes for every phone, a smaller state for the admin and the board
Network emulator	Admin network settings	Added delay, jitter and loss for each player and for the board
Ping and metrics	Ping replies, order numbers	Ping, jitter, loss, clock offset and frame rate for the admin graphs
Board link	Board messages	One TCP channel and one UDP stream to the board

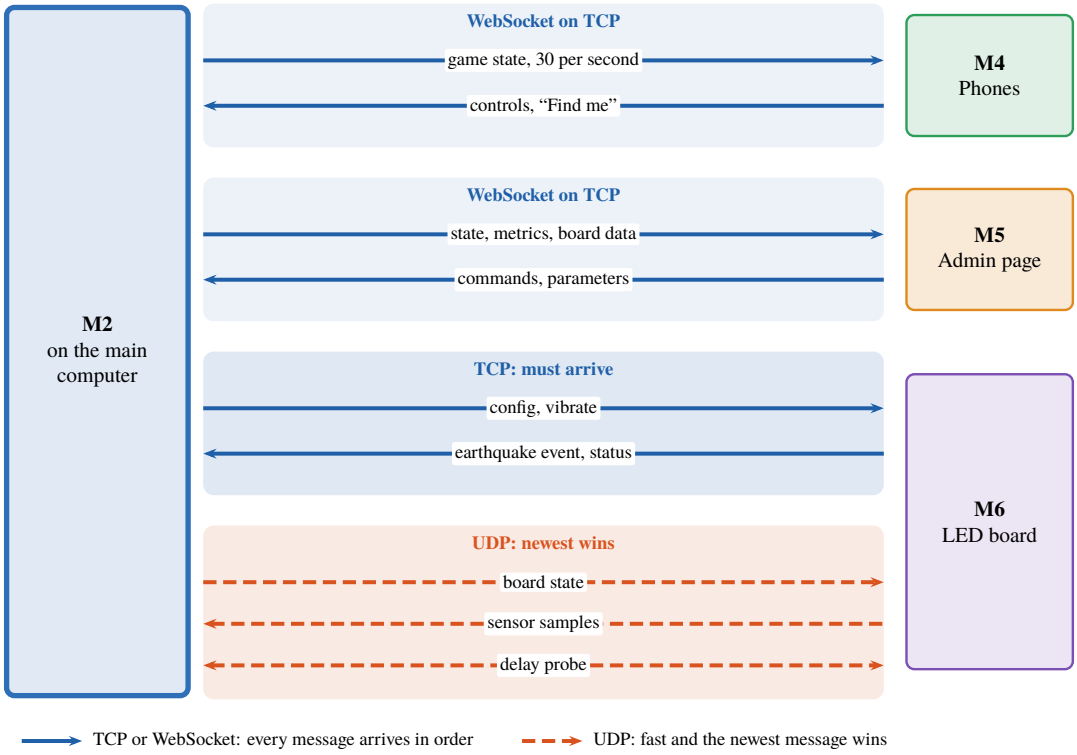
5. How it works

Which link uses which protocol:

A web browser can only open a WebSocket for talk in both directions, so phones and the admin page use WebSocket. WebSocket runs on TCP, so every message arrives and the order is kept. The LED board is not a browser, so it can use both TCP and UDP. On TCP it gets the messages that must never be lost: the config (player colours, brightness, power limit) and the vibrate commands. The board also sends earthquake events and its status report on TCP. The fast and repeated data goes on UDP: the board state, the sensor samples and the small delay probes. Here only the newest message is useful. If one UDP packet is lost, the next one replaces it. TCP would wait for the lost packet and the LEDs would stop for a moment, so UDP is the better choice for this data.

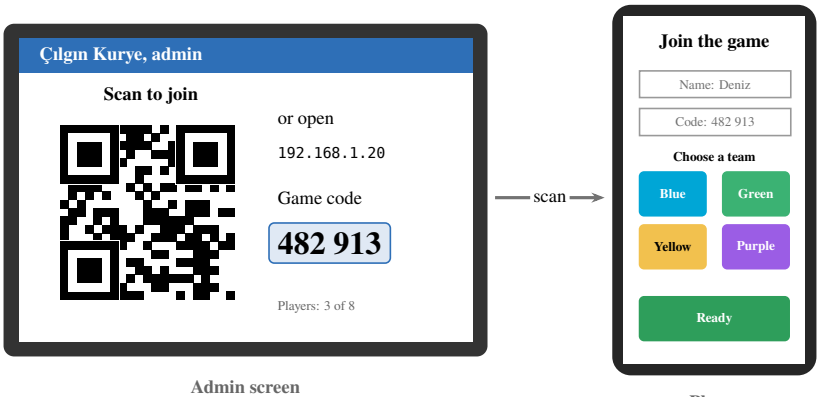
Messages on the board TCP channel:

WebSocket cuts the data into messages by itself, but plain TCP is only a stream of bytes. So each message on the board TCP channel starts with its length, and the other side reads exactly that many bytes. Small messages like a vibrate command are sent at once, without waiting to fill a bigger packet. Every message on both board channels has an order number, so old UDP packets can be dropped. Each start of the server makes a new session number. When the board sees a new session number, it starts its order numbers again, so new messages are not dropped as old ones.



Joining:

At the start the admin screen shows a QR code and a six digit game code. The QR code holds the IP address of the main computer and the game code. A player scans it, and the phone opens the game page from the main computer. A player without a camera types the address and the code. The first time, the browser saves a random player key. If the page reloads or Wi-Fi drops, the phone sends the same key and gets the same motorcycle back. The LED board first looks for the server on the local network. If it finds nothing, it uses a backup server address saved in the ESP32’s flash memory.



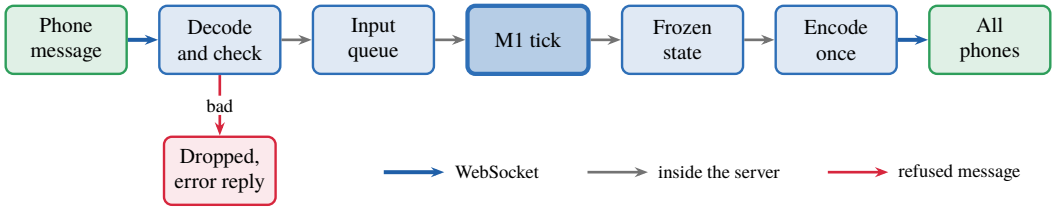
Admin screen Phone

Keeping the game state safe:

Many phones send messages at the same time, but only M1 may change the game state, and only inside a tick. So M2 never writes to the state. A phone can only send controls. It can never send a position or a score. M2 decodes each message, checks its type and fields, and keeps every value in its range (for example steer stays between −1 and 1). Each session can only control its own player. A message with an old or repeated order number is dropped. Each session also has a limit on the number and size of its messages, so one phone cannot flood the server. Good inputs wait in a queue, and M1 reads the queue at the start of the next tick. At the end of the tick M2 encodes the frozen state once and sends the same bytes to all phones.

Network settings from the admin:

For tests the admin can make the network worse by hand. Each player has an up queue and a down queue, and the board link has the same pair. The admin sets three values for each queue: an added delay, a jitter that makes the delay change randomly, and a loss chance. The loss is used only on the state and control messages. Join, “Find me” and admin replies are never dropped, because the game would break without them. The admin can also set a frame rate limit for a phone and the board state rate, from 10 to 30 per second.



Measuring the network:

Twice per second M2 sends a ping with the server time. The phone answers with its own time, its frame rate and the number of the last state it got. From this M2 finds the round trip time, the jitter and the clock offset of the phone. On WebSocket a real Wi-Fi loss is not seen as a missing message, because TCP sends it again. It shows as a jump in the delay. So for phones M2 reports ping and jitter. On the board UDP stream a lost packet is really missing, so gaps in the order numbers give the real packet loss.

How much data:

We expect the full game state to be a few kilobytes. Sent 30 times per second to 8 phones, this is only a few megabits per second in total, which a normal 2.4 GHz Wi-Fi router can carry.

6. Testing

M2 is tested alone with test data instead of the real neighbours. A fake M1 sends a moving test state, scripted bot phones play the role of up to 8 real phones, and a small test program plays the board. With this setup we send good, broken and too large messages and check that no bad value reaches M1. We also check the load with 8 phones, how exact the added delay, jitter and loss are, that join and “Find me” are never dropped, reconnecting with the player key, and a server restart with the test board.